

INT202
Complexity of Algorithms
Minimum Spanning Tree and
Network Flow

Xi'an Jiaotong-Liverpool University
2025-2026

Minimum Spanning Tree(MST)

Let G be an undirected graph.

We are also often interested in finding a spanning tree T , i.e. an acyclic spanning subgraph of G , which *minimizes* the sum of the weights of the edges of T .

Computing a spanning tree T with smallest total weight is the problem of constructing a *Minimum Spanning Tree* or MST for short.

Prim's Algorithm

Prim's algorithm is a greedy type of algorithm designed to find MST.

The idea is that we build the MST starting at a root vertex.

At each step, we “grow” the tree by adding an additional edge to a vertex not in the current tree, selecting one that minimizes the “attachment cost” while ensuring that we do not create a cycle with the edges already selected.

Prim's Algorithm

The pseudocode for prim's algorithm shows how we create two sets of vertices U and $V-U$. U contains the list of vertices that have been visited and $V-U$ the list of vertices that haven't. One by one, we move vertices from set $V-U$ to set U by connecting the least weight edge.

$T = \emptyset$;

$U = \{ 1 \}$;

while ($U \neq V$)

 let (u, v) be the lowest cost edge such that $u \in U$ and $v \in V - U$;

$T = T \cup \{(u, v)\}$;

$U = U \cup \{v\}$

Prim's Algorithm

$T = \emptyset;$

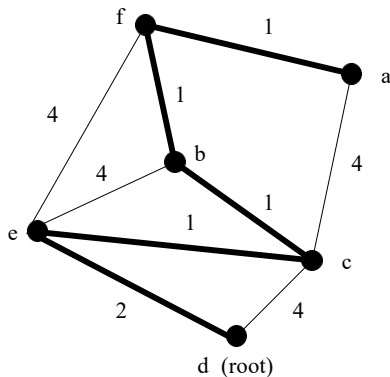
$U = \{r\};$

while ($U \neq V$)

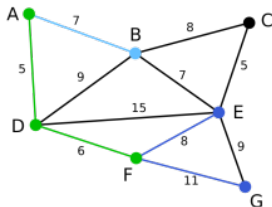
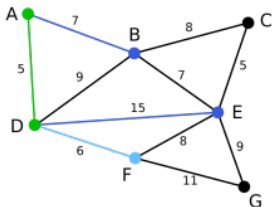
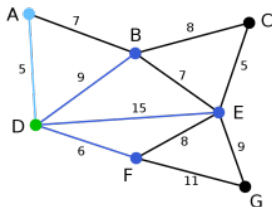
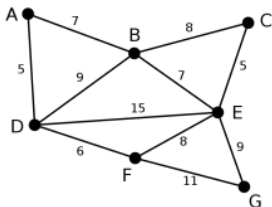
let (u, v) be the lowest cost edge such that $u \in U$ and $v \in V - U$;

$T = T \cup \{(u, v)\};$

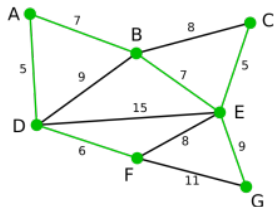
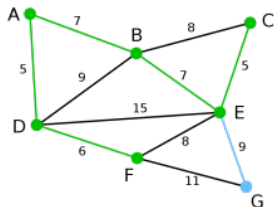
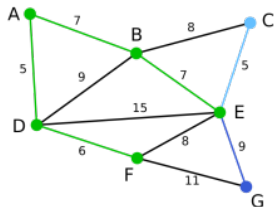
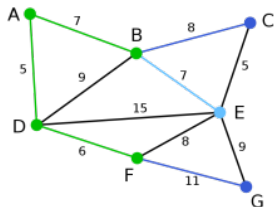
$U = U \cup \{v\}$



Prim's Algorithm



Prim's Algorithm



Kruskal's Algorithm for MST

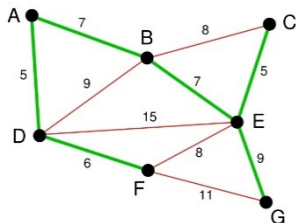
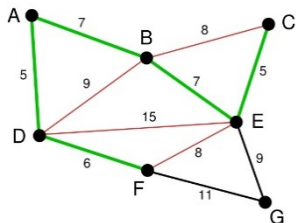
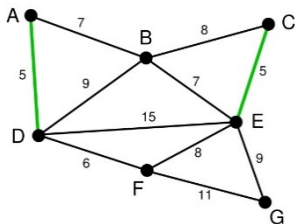
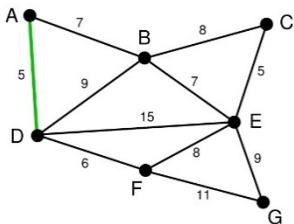
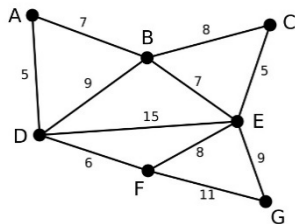
Kruskal's algorithm, like Prim's, is a greedy algorithm.

Unlike Prim's which “grows” a MST starting from some vertex, Kruskal's algorithm works by considering the edges in order from smallest to largest. At each step, we greedily pick the smallest edge available that does not create a cycle together with those chosen so far.

The steps for implementing Kruskal's algorithm are as follows:

1. Sort all the edges from low weight to high;
2. Take the edge with the lowest weight and add it to the spanning tree. If adding the edge created a cycle, then reject this edge;
3. Keep adding edges until we reach all vertices.

Kruskal's Algorithm for MST

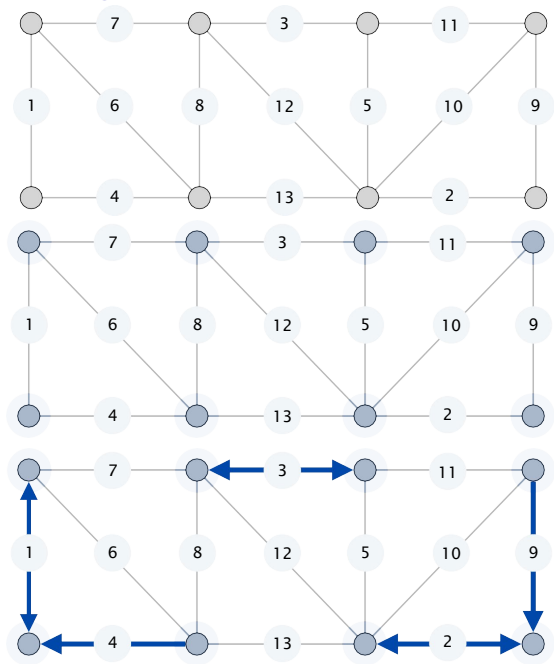


Borůvka's algorithm

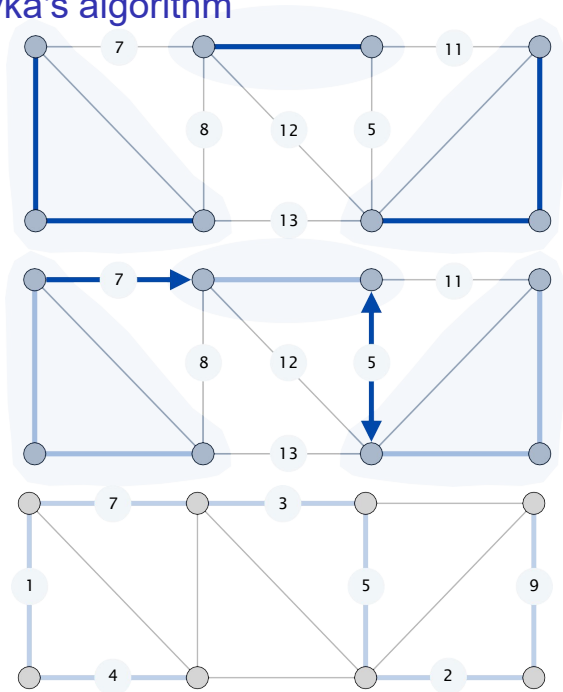
Another greedy algorithm to find the minimum spanning tree for a graph:

- 1) Input is a connected, weighted and un-directed graph.
- 2) Initialize all vertices as individual components.
- 3) Initialize MST as empty.
- 4) While there are more than one components, do following for each component.
 - a) Find the closest weight edge that connects this component to any other component.
 - b) Add this closest edge to MST if not already added.
- 5) Return MST.

Borůvka's algorithm

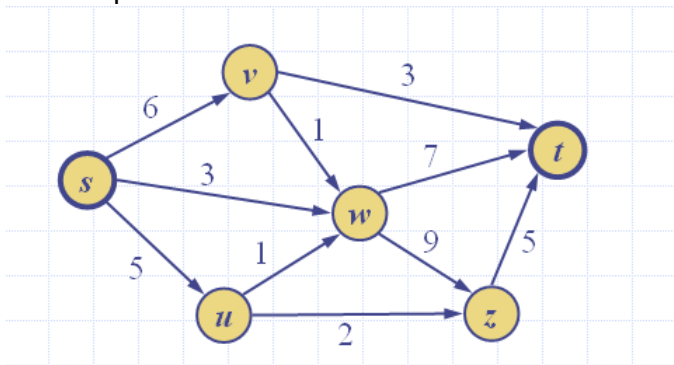


Borůvka's algorithm



Flow Network

- ▶ A flow network (or just network) N consists of
 - ▶ A weighted digraph G with nonnegative integer edge weights, where the weight of an edge e is called the capacity $c(e)$ of e .
 - ▶ Two distinguished vertices, s and t of G , called the *source* and *sink*, respectively, such that s has no incoming edges and t has no outgoing edges.
- ▶ An example



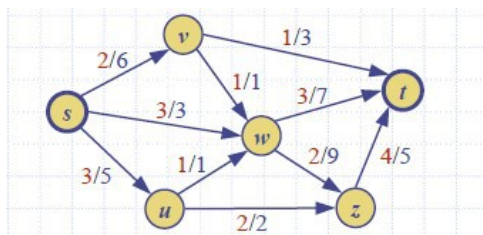
Flow

- ▷ A flow f for a network N is an assignment of an integer value $f(e)$ to each edge e that satisfies the following properties:

1. Capacity Rule: For each edge e , $0 \leq f(e) \leq c(e)$
2. Conservation Rule: For each vertex $v \neq s, t$

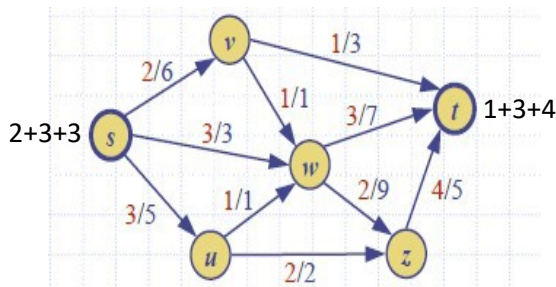
$$\sum_{e \in E^-(v)} f(e) = \sum_{e \in E^+(v)} f(e)$$

where $E^-(v)$ and $E^+(v)$ are the incoming and outgoing edges of v , respectively.



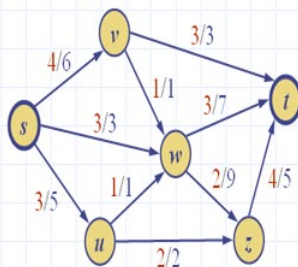
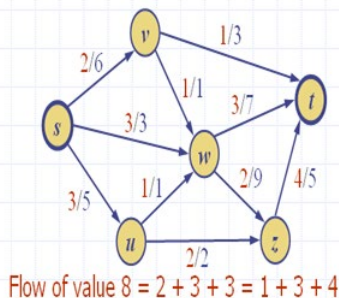
Flow

- ▶ The value of a flow f , denoted $|f|$, is the total flow from the source, which is the same as the total flow into the sink.
- ▶ Problem: Find a flow of maximum value?



Maximum Flow

- ▶ A flow for a network N is said to be maximum if its value is the largest of all flows for N .
- ▶ The maximum flow problem consists of finding a maximum flow for a given network N

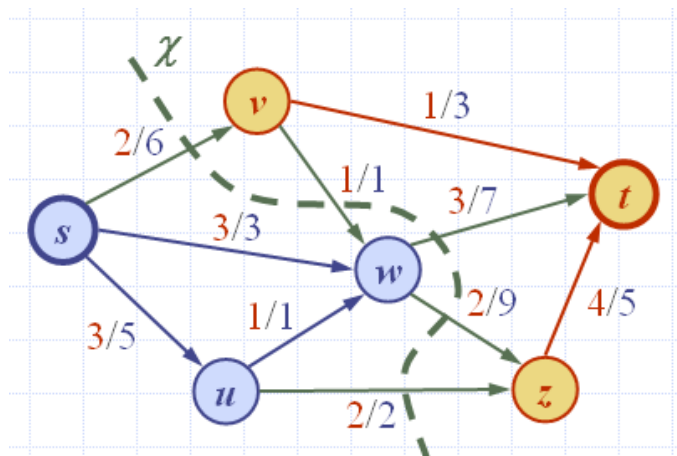


Cut - Definitions

Flows are closely related to another concept, known as *cuts*.

- ▶ A *cut* of a network N with source s and sink t is a partition $\chi = (V_s, V_t)$ of the vertices of N such that $s \in V_s$ and $t \in V_t$.
 - ▶ Forward edge of cut χ : origin in V_s and destination in V_t
 - ▶ Backward edge of cut χ : origin in V_t and destination in V_s
- ▶ Flow $f(\chi)$ across a cut χ : total flow of forward edges minus total flow of backward edges
- ▶ Capacity $c(\chi)$ of a cut χ : total capacity of forward edges

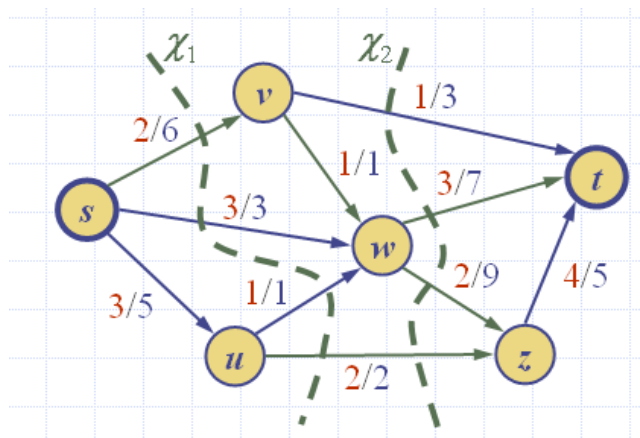
Cut - Example



The *total flow/capacity* of the cut are

- ▷ $f(\chi) = 8$
- ▷ $c(\chi) = 24$

Flow and Cut



The *total flow/capacity* of the cut are

- ▷ $c(\chi_1) = 12 = 6 + 3 + 1 + 2$
- ▷ $c(\chi_2) = 21 = 3 + 7 + 9 + 2$
- ▷ For both cuts, the value of the flow is $|f| = 8$

Flow and Cut

Lemma 1: Let N be a flow network, and let f be a flow for N . For any cut χ of N , the value of f is equal to the flow across cut χ , that is, $|f| = f(\chi)$.

Lemma 2: Let N be a flow network, and let χ be a cut of N . Given any flow f for N , the flow across cut does not exceed the capacity of χ , that is, $f(\chi) \leq c(\chi)$.

Theorem 1: The value of any flow is less than or equal to the capacity of any cut, i.e., for any flow f and any cut χ , we have $|f| \leq c(\chi)$.

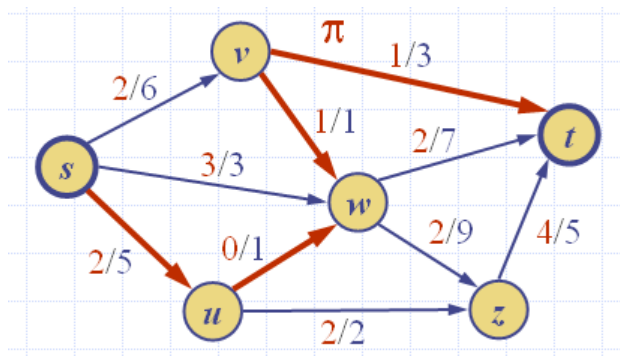
$\Rightarrow$ The value of a maximum flow is no more than the capacity of a minimum cut.

Augmenting Path - Definitions

To construct a maximum flow we need the concept of ***augmenting path***.

- ▶ Consider a flow f for a network N
- ▶ Let e be an edge from u to v :
 - ▶ *Residual capacity* of e from u to v : $\Delta_f(u, v) = c(e) - f(e)$
 - ▶ *Residual capacity* of e from v to u : $\Delta_f(v, u) = f(e)$
- ▶ Let π be a path from s to t , the *residual capacity* $\Delta_f(\pi)$ of π is the smallest of the residual capacities of the edges of π in the direction from s to t .
- ▶ A path π from s to t is an augmenting path if $\Delta_f(\pi) > 0$

Augmenting Path - Example



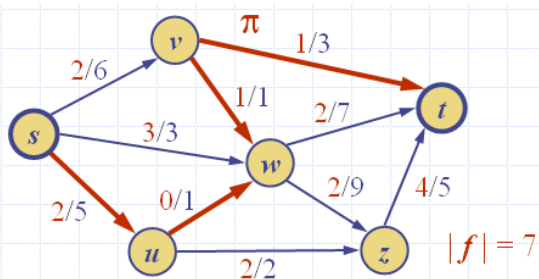
We get the following residual capacities and flow

- ▷ $\Delta_f(s, u) = 3, \Delta_f(u, w) = 1, \Delta_f(w, v) = 1, \Delta_f(v, t) = 2.$
- ▷ $\Delta_f(n) = 1$
- ▷ $|f| = 7$

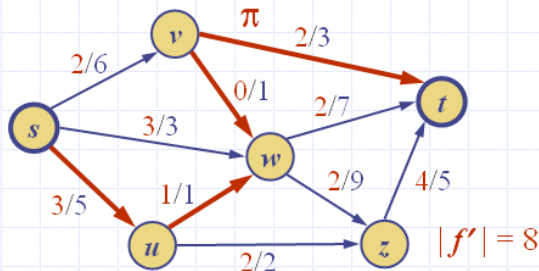
Flow Augmentation

We can always add the residual capacity of an augmenting path to an existing flow and get another valid flow.

Lemma 3: Let π be an augmenting path for flow f in network N . There exists a flow f' for N of value $|f'| = |f| + \Delta_f(\pi)$



$$\Downarrow \Delta_f(\pi) = 1$$



Flow Augmentation

What if there is no augmenting path for a flow f in network N ?

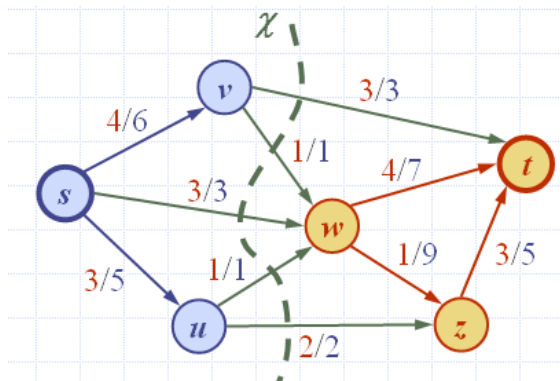
Lemma 4: If a network N does not have an augmenting path with respect to a flow f , then f is a maximum flow. Also, there is a cut χ of N such that $|f| = c(\chi)$

Lemma 4 and Theorem 1 gives us the Max-flow, Min-Cut Theorem also known as the *Ford-Fulkerson Theorem*

Theorem 2: The value of a maximum flow is equal to the capacity of a minimum cut.

Max-Flow and Min-Cut

- ▷ Define
 - ▷ V_s set of vertices reachable from s by augmenting paths
 - ▷ V_t set of remaining vertices
- ▷ Cut $\chi = (V_s, V_t)$ has capacity $c(\chi) = |f|$
 - ▷ Forward edge: $f(e) = c(e)$
 - ▷ Backward edge: $f(e) = 0$
- ▷ Thus, flow f has maximum value and cut χ has minimum capacity. Below we have $c(\chi) = |f| = 10$.

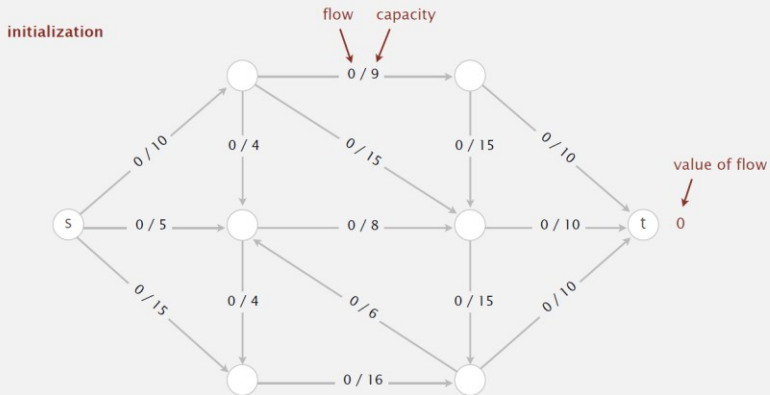


The Ford-Fulkerson Algorithm

- ▷ Initially, $f(e) = 0$ for each edge e
- ▷ Repeatedly
 - ▷ Search for an augmenting path π
 - ▷ Compute bottleneck capacity $\Delta_f(\pi)$
 - ▷ Increase flow along the edges of π by bottleneck capacity $\Delta_f(\pi)$

Ford-Fulkerson algorithm

Initialization. Start with 0 flow.

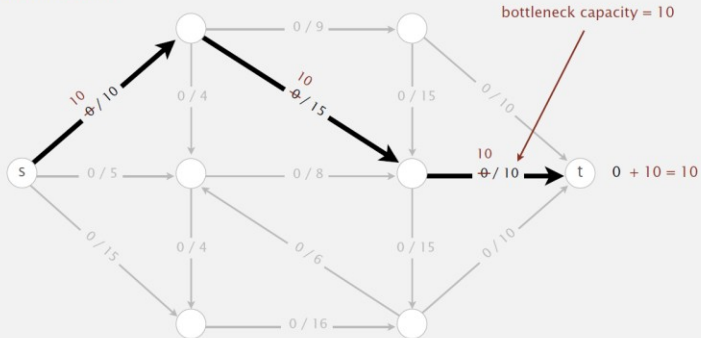


Idea: increase flow along augmenting paths

Augmenting path. Find an undirected path from s to t such that:

- Can increase flow on forward edges (not full).
- Can decrease flow on backward edge (not empty).

1st augmenting path

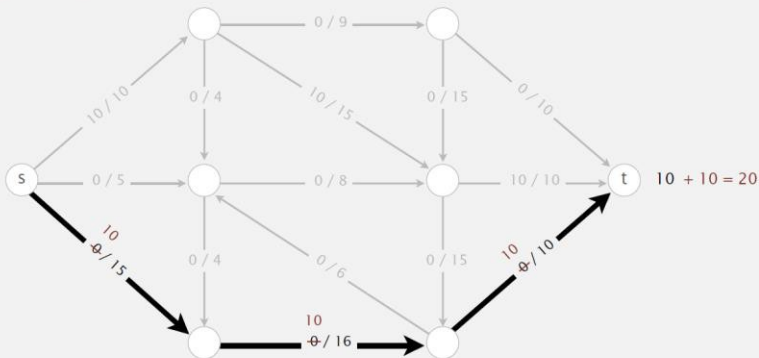


Idea: increase flow along augmenting paths

Augmenting path. Find an undirected path from s to t such that:

- Can increase flow on forward edges (not full).
- Can decrease flow on backward edge (not empty).

2nd augmenting path

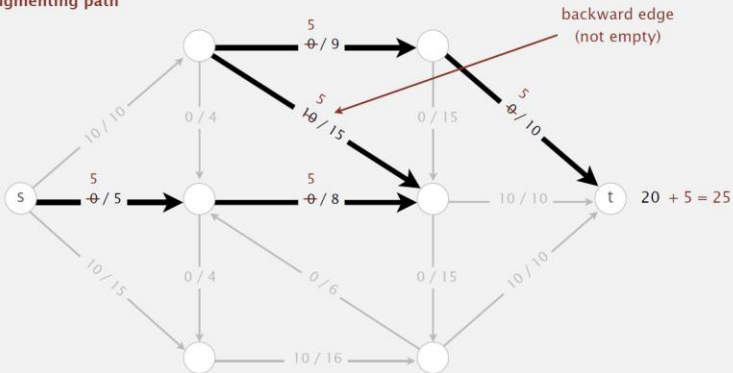


Idea: increase flow along augmenting paths

Augmenting path. Find an undirected path from s to t such that:

- Can increase flow on forward edges (not full).
- Can decrease flow on backward edge (not empty).

3rd augmenting path

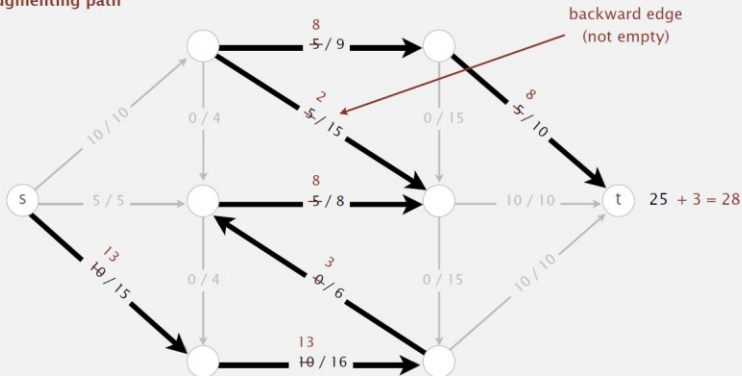


Idea: increase flow along augmenting paths

Augmenting path. Find an undirected path from s to t such that:

- Can increase flow on forward edges (not full).
- Can decrease flow on backward edge (not empty).

4th augmenting path

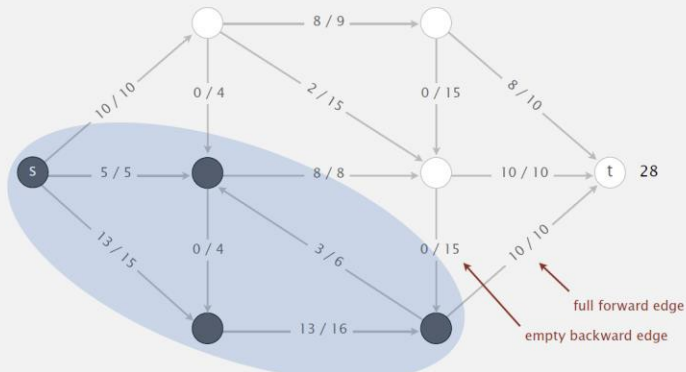


Idea: increase flow along augmenting paths

Termination. All paths from s to t are blocked by either a

- Full forward edge.
- Empty backward edge.

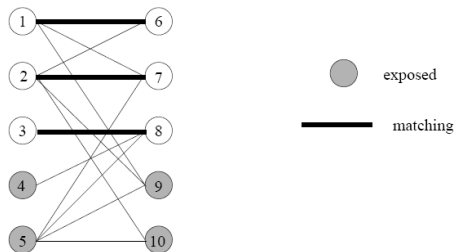
no more augmenting paths



Bipartite Matching - Definitions

- A graph $G = (V, E)$ is called *bipartite* if and only if
 - The set V can be partitioned into two sets, X and Y .
 - Every edge of G has an endpoint in X and the other endpoint in Y .
- A *matching* $M \subseteq E$ is a set of edges that have no endpoints in common. Each vertex from one set has at most one "partner" in the other set.
- If a vertex v has no edge of M incident to it then v is said to be *exposed* (or *unmatched*)).
- A matching is *perfect* if no vertex is exposed.

Bipartite Matching - Example



The maximum bipartite matching problem is to find a matching M with the greatest number of edges (over all matchings).

Maximum Bipartite Matching

Maximum Bipartite Matching

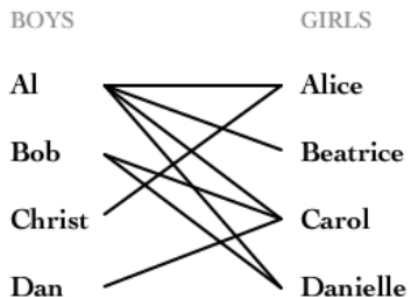
Given a bipartite graph $G = (A \cup B, E)$, find an $S \subseteq A \times B$ that is a matching and is as large as possible.

Notes:

- We're given A and B so we don't have to find them.
- S is a **perfect matching** if every vertex is matched.

Maximum Bipartite Matching

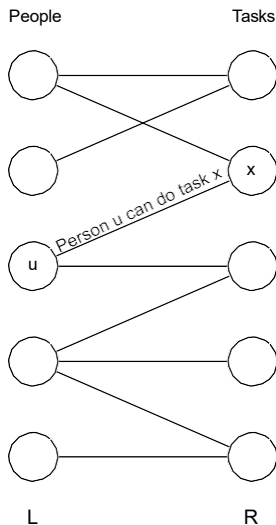
Example 1: Let G be a bipartite graph where the set X represents a group of boys and the set Y a group of girls, who are all together at a community dance. Let there be an edge joining $x \in X$ and $y \in Y$ if x and y are willing to dance with one another. A maximum matching in G corresponds to a largest set of compatible pairs of boy and girl who can all happily dancing at the same time.



Maximum Bipartite Matching

Example 2: Let G be a bipartite graph. Suppose we have a set of people L and set of jobs R .

- Each person can do only some of the jobs.
- Can model this as a bipartite graph



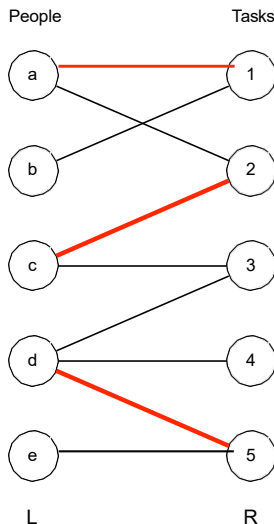
Maximum Bipartite Matching

- A **matching** gives an assignment of people to tasks.

If a vertex v has no edge of M incident to it then v is said to be **exposed (or unmatched)**.

A **maximal bipartite matching** is the largest subset of edges in a bipartite graph such that no two selected edges share a common vertex.

- (This one is not maximum.)



(a,1), (c,2), (d,5) are matched.
b, e, 3, 4 are exposed.

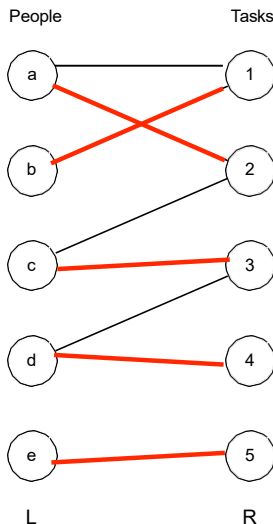
Maximum Bipartite Matching

- A **matching** gives an assignment of people to tasks.

If a vertex v has no edge of M incident to it then v is said to be **exposed (or unmatched)**.

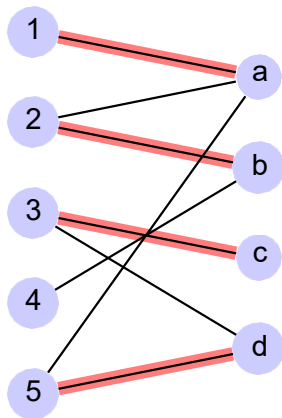
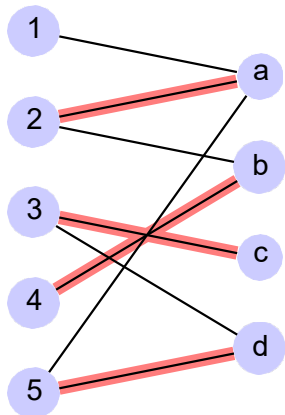
A **maximal bipartite matching** is the largest subset of edges in a bipartite graph such that no two selected edges share a common vertex.

A matching is **perfect** if no vertex is exposed.



Maximum Bipartite Matching

Matching not guaranteed to be unique.

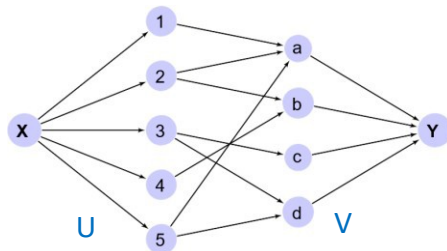


Reduce

We can solve the maximum bipartite matching problem using a *network flow* approach.

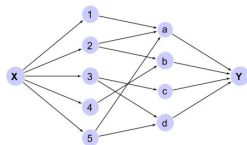
- We first ensure that all edges from U to V are directed.
- We introduce a single **source** node that has outgoing edges to each node in U
- We then add a **sink** node that has incoming edges from each node in V .

The problem now becomes finding the *maximal flow* in the graph.



We denote the source node as **X** and the sink node as **Y**

Reduce



One can more intuitively consider flow as water running through pipes represented by the edges.

There are many algorithms one could use to solve *network flow* problems (such as Ford–Fulkerson algorithm). For the specific case of **bipartite graphs**, we don't need the full generality of most network flow algorithms.

Since we've converted the problem to an unweighted directed graph, we can therefore use the following simplified algorithm which solves the max flow problem for any general unweighted directed graph.

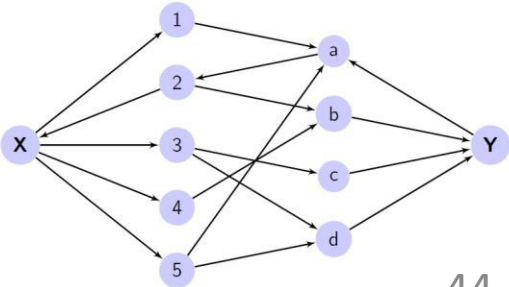
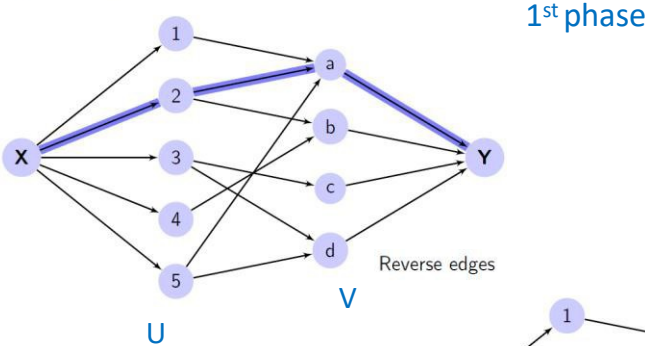
Maximum Bipartite Matching

Solving maximal flow for an unweighted directed graph:

1. Construct the directed graph with a source and sink node.
2. Using a search method to find a path from the source to the sink.
3. Once a path is found, reverse each edge on this path.
4. Repeat step 2 and 3 until no more paths from the sink to source exist.
5. The final matching solution is then the set of edges between U and V that are reversed (i.e. run from V to U).

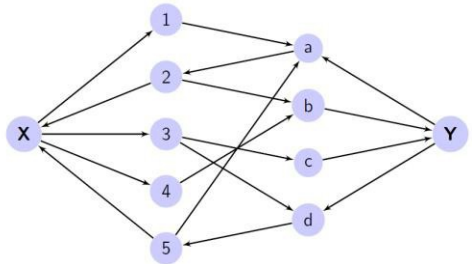
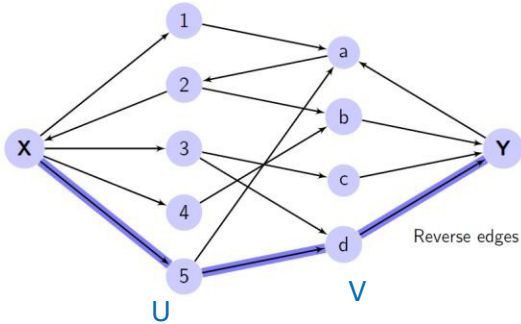
Maximum Bipartite Matching

Found path: $X \rightarrow 2 \rightarrow a \rightarrow Y$



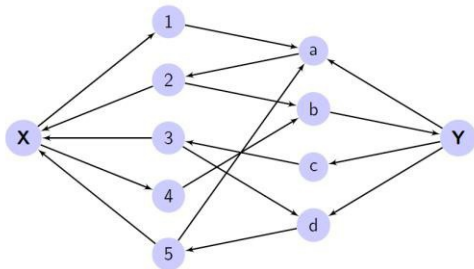
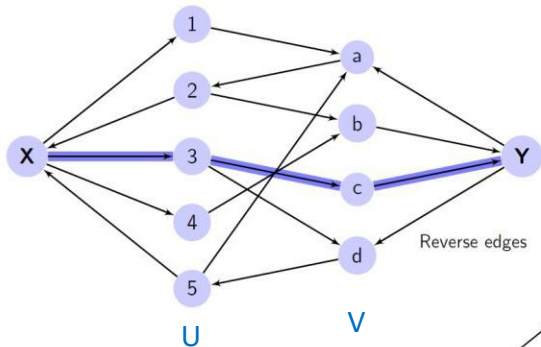
Maximum Bipartite Matching

Found path: $X \rightarrow 5 \rightarrow d \rightarrow Y$



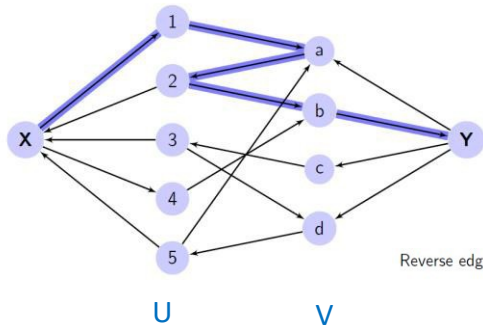
Maximum Bipartite Matching

Found path: $X \rightarrow 3 \rightarrow c \rightarrow Y$

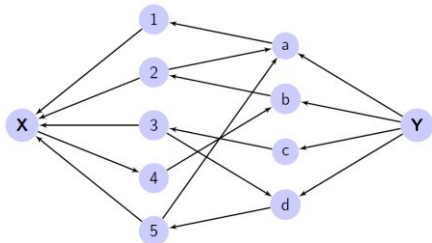


Maximum Bipartite Matching

Found path: $X \rightarrow 1 \rightarrow a \rightarrow 2 \rightarrow b \rightarrow Y$



Reverse edges



Maximum Bipartite Matching

No more paths found. Matching is reversed edges.

